

OOP 1 (C++) Weekly Assignments

Complete Solutions

This PDF contains separate C++ programs for the tasks in the uploaded “Weekly Assignments_OOP1” document. The solutions follow the wording and requirements in the assignment. Where the assignment does not specify a rate/value, the program asks the user to enter it.

Important: Save each program as its own .cpp file, compile it, and test it before submission. The uploaded source is a scanned document, so a few class names/words were unclear in OCR; the intended names are used where the surrounding task makes them clear.

Week 1 — Task 1

Mobile Phone Sales Receipt System

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    string customerName, phoneModel;
    int quantity;
    double pricePerPhone, totalSales;

    cout << "Enter customer name: ";
    getline(cin, customerName);

    cout << "Enter phone model: ";
    getline(cin, phoneModel);

    cout << "Enter quantity bought: ";
    cin >> quantity;

    cout << "Enter price per phone: ";
    cin >> pricePerPhone;

    totalSales = quantity * pricePerPhone;

    cout << "\n===== SALES RECEIPT =====\n";
    cout << "Customer Name : " << customerName << '\n';
    cout << "Phone Model   : " << phoneModel << '\n';
    cout << "Quantity      : " << quantity << '\n';
    cout << fixed << setprecision(2);
    cout << "Price/Phone   : " << pricePerPhone << '\n';
    cout << "Total Sales   : " << totalSales << '\n';
    cout << "===== \n";

    return 0;
}
```

Week 1 — Task 2

Driving Test Result Evaluation System

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    string studentName;
    double theoryMarks, practicalMarks, averageScore;

    cout << "Enter student name: ";
    getline(cin, studentName);

    cout << "Enter theory test marks: ";
    cin >> theoryMarks;

    cout << "Enter practical test marks: ";
    cin >> practicalMarks;

    averageScore = (theoryMarks + practicalMarks) / 2.0;

    cout << "\n===== DRIVING TEST RESULT =====\n";
    cout << "Student      : " << studentName << '\n';
    cout << "Theory Marks   : " << theoryMarks << '\n';
    cout << "Practical     : " << practicalMarks << '\n';
    cout << fixed << setprecision(2);
    cout << "Average Score : " << averageScore << '\n';

    if (averageScore >= 50)
        cout << "Result      : PASSED\n";
    else
        cout << "Result      : FAILED\n";

    return 0;
}
```

Week 2 — Task 1

Grading System (if-else ladder)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string studentName;
    double marks;
    char grade;

    cout << "Enter student name: ";
    getline(cin, studentName);

    cout << "Enter exam marks (0-100): ";
    cin >> marks;

    if (marks >= 70 && marks <= 100)
        grade = 'A';
    else if (marks >= 60)
        grade = 'B';
    else if (marks >= 50)
        grade = 'C';
    else if (marks >= 40)
        grade = 'D';
    else if (marks >= 0)
        grade = 'E';
    else {
        cout << "Invalid marks.\n";
        return 0;
    }

    cout << "\nStudent : " << studentName << '\n';
    cout << "Marks    : " << marks << '\n';
    cout << "Grade     : " << grade << '\n';

    return 0;
}
```

Week 2 — Task 2

Admission Decision (Nested if)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string studentName;
    int age;
    double score;

    cout << "Enter student name: ";
    getline(cin, studentName);

    cout << "Enter age: ";
    cin >> age;

    cout << "Enter exam score: ";
    cin >> score;

    cout << "\nStudent: " << studentName << '\n';

    if (age >= 18) {
        if (score >= 50)
            cout << "Decision: Admitted\n";
        else
            cout << "Decision: Not Admitted: Low Score\n";
    } else {
        cout << "Decision: Not Admitted: Underage\n";
    }

    return 0;
}
```

Week 2 — Task 3

Simple Calculator (switch statement)

```
#include <iostream>
using namespace std;

int main() {
    double number1, number2, result;
    char operation;

    cout << "Enter first number: ";
    cin >> number1;

    cout << "Enter operator (+, -, *, /): ";
    cin >> operation;

    cout << "Enter second number: ";
    cin >> number2;

    switch (operation) {
        case '+':
            result = number1 + number2;
            cout << "Result = " << result << '\n';
            break;

        case '-':
            result = number1 - number2;
            cout << "Result = " << result << '\n';
            break;

        case '*':
            result = number1 * number2;
            cout << "Result = " << result << '\n';
            break;

        case '/':
            if (number2 == 0)
                cout << "Error: Division by zero is not allowed.\n";
            else {
                result = number1 / number2;
                cout << "Result = " << result << '\n';
            }
            break;

        default:
            cout << "Invalid operator.\n";
    }

    return 0;
}
```

Week 3 — Task 1

Payroll System Using Functions

Note: The assignment does not provide a numeric overtime rate per hour, so this solution asks the user to enter it.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

string employeeName;
double basicSalary, overtimeHours, ratePerHour;
double overtimePay, netSalary;

void getEmployeeDetails() {
    cout << "Enter employee name: ";
    getline(cin, employeeName);

    cout << "Enter basic salary: ";
    cin >> basicSalary;

    cout << "Enter overtime hours: ";
    cin >> overtimeHours;

    // The assignment gives the formula but not the rate, so input it.
    cout << "Enter overtime rate per hour: ";
    cin >> ratePerHour;
}

void calculateOvertimePay() {
    overtimePay = overtimeHours * ratePerHour;
}

void calculateNetSalary() {
    netSalary = basicSalary + overtimePay;
}

void displayPayslip() {
    cout << fixed << setprecision(2);
    cout << "\n===== PAYSIP =====\n";
    cout << "Employee Name   : " << employeeName << '\n';
    cout << "Basic Salary    : " << basicSalary << '\n';
    cout << "Overtime Hours  : " << overtimeHours << '\n';
    cout << "Overtime Pay    : " << overtimePay << '\n';
    cout << "Net Salary      : " << netSalary << '\n';
    cout << "=====\\n";
}

int main() {
    getEmployeeDetails();
    calculateOvertimePay();
    calculateNetSalary();
    displayPayslip();

    return 0;
}
```

Week 3 — Task 2

Water Billing System Using Functions

Note: The assignment does not provide a rate per unit, so this solution asks the user to enter it.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

string customerName;
double unitsConsumed, ratePerUnit;
double billBeforeDiscount, discount, finalAmount;

void getCustomerDetails() {
    cout << "Enter customer name: ";
    getline(cin, customerName);

    cout << "Enter units consumed: ";
    cin >> unitsConsumed;

    // The assignment does not state the rate, so input it.
    cout << "Enter rate per unit: ";
    cin >> ratePerUnit;
}

void calculateBill() {
    billBeforeDiscount = unitsConsumed * ratePerUnit;
}

void applyDiscount() {
    if (unitsConsumed > 100)
        discount = 0.10 * billBeforeDiscount;
    else
        discount = 0;

    finalAmount = billBeforeDiscount - discount;
}

void displayBill() {
    cout << fixed << setprecision(2);
    cout << "\n===== WATER BILL =====\n";
    cout << "Customer Name      : " << customerName << '\n';
    cout << "Units Consumed       : " << unitsConsumed << '\n';
    cout << "Bill Before Discount: " << billBeforeDiscount << '\n';
    cout << "Discount             : " << discount << '\n';
    cout << "Final Amount Payable: " << finalAmount << '\n';
    cout << "===== \n";
}

int main() {
    getCustomerDetails();
    calculateBill();
    applyDiscount();
    displayBill();

    return 0;
}
```

Week 4 — Task 1

Employee Bonus System (for loop)

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    string employeeName;
    double basicSalary, bonus, totalSalary;

    for (int i = 1; i <= 5; i++) {
        cout << "\nEmployee " << i << '\n';

        cout << "Enter name: ";
        cin >> ws;
        getline(cin, employeeName);

        cout << "Enter basic salary: ";
        cin >> basicSalary;

        bonus = 0.05 * basicSalary;
        totalSalary = basicSalary + bonus;

        cout << fixed << setprecision(2);
        cout << "Name      : " << employeeName << '\n';
        cout << "Basic Salary: " << basicSalary << '\n';
        cout << "Bonus       : " << bonus << '\n';
        cout << "Total Salary: " << totalSalary << '\n';
    }

    return 0;
}
```

Week 4 — Task 2

Savings Withdrawal System (while loop)

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double balance, withdrawal;

    cout << "Enter account balance: ";
    cin >> balance;

    while (balance > 0) {
        cout << "\nEnter withdrawal amount (0 to stop): ";
        cin >> withdrawal;

        if (withdrawal <= 0) {
            cout << "Withdrawal stopped.\n";
            break;
        }

        if (withdrawal > balance) {
            cout << "Withdrawal amount is greater than the balance.\n";
            break;
        }

        balance -= withdrawal;

        cout << fixed << setprecision(2);
        cout << "Withdrawal successful.\n";
        cout << "Remaining balance: " << balance << '\n';

        if (balance == 0)
            cout << "Balance is zero. Transaction ended.\n";
    }

    return 0;
}
```

Week 4 — Task 3

Password Verification System (do-while loop)

Note: The assignment asks for predefined credentials but does not specify their values. This solution uses username 'admin' and password 'password123'.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const string correctUsername = "admin";
    const string correctPassword = "password123";

    string username, password;

    do {
        cout << "Enter username: ";
        cin >> username;

        cout << "Enter password: ";
        cin >> password;

        if (username == correctUsername && password == correctPassword) {
            cout << "Access Granted\n";
            break;
        } else {
            cout << "Incorrect credentials, try again\n\n";
        }
    } while (true);

    return 0;
}
```

Week 5

Hotel Management System Using Arrays

Note: Occupancy is simulated with random 0/1 values as permitted by the assignment.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    // ----- 1D Array: Weekly Revenue -----
    double revenue[7];
    double totalRevenue = 0;

    cout << "Enter revenue for 7 days:\n";
    for (int i = 0; i < 7; i++) {
        cout << "Day " << i + 1 << ": ";
        cin >> revenue[i];
        totalRevenue += revenue[i];
    }

    double averageRevenue = totalRevenue / 7.0;

    cout << "\nWeekly Revenue = " << totalRevenue << '\n';
    cout << "Average Daily Revenue = " << averageRevenue << '\n';

    // ----- 2D Array: One Branch -----
    int occupancy[5][10];
    int occupied, vacant;

    srand(static_cast<unsigned int>(time(nullptr)));

    cout << "\nRoom Occupancy Per Floor\n";
    for (int floor = 0; floor < 5; floor++) {
        occupied = 0;
        vacant = 0;

        for (int room = 0; room < 10; room++) {
            occupancy[floor][room] = rand() % 2;

            if (occupancy[floor][room] == 1)
                occupied++;
            else
                vacant++;
        }

        cout << "Floor " << floor + 1
              << " -> Occupied: " << occupied
              << ", Vacant: " << vacant << '\n';
    }

    // ----- 3D Array: Three Branches -----
    int chain[3][5][10];
    int totalOccupied = 0;

    for (int branch = 0; branch < 3; branch++) {
        for (int floor = 0; floor < 5; floor++) {
            for (int room = 0; room < 10; room++) {
                chain[branch][floor][room] = rand() % 2;

                if (chain[branch][floor][room] == 1)
                    totalOccupied++;
            }
        }
    }

    cout << "\nTotal occupied rooms across all branches: "
          << totalOccupied << '\n';

    return 0;
}
```

Week 6 — Task 1

Library Book Management System

```
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int copiesAvailable;

public:
    void inputDetails() {
        cout << "Enter book title: ";
        getline(cin, title);

        cout << "Enter author: ";
        getline(cin, author);

        cout << "Enter number of copies available: ";
        cin >> copiesAvailable;
    }

    void borrowBook() {
        if (copiesAvailable > 0) {
            copiesAvailable--;
            cout << "Book borrowed successfully.\n";
        } else {
            cout << "No copies available.\n";
        }
    }

    void displayDetails() {
        cout << "\n===== BOOK DETAILS =====\n";
        cout << "Title          : " << title << '\n';
        cout << "Author           : " << author << '\n';
        cout << "Copies Available : " << copiesAvailable << '\n';
    }
};

int main() {
    Book book;

    book.inputDetails();
    book.borrowBook();
    book.displayDetails();

    return 0;
}
```

Week 6 — Task 2

Student Fee Management System

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Student {
private:
    string studentName;
    string admissionNumber;
    double feeBalance;

public:
    void inputStudent() {
        cout << "Enter student name: ";
        getline(cin, studentName);

        cout << "Enter admission number: ";
        getline(cin, admissionNumber);

        cout << "Enter fee balance: ";
        cin >> feeBalance;
    }

    void makePayment() {
        double payment;

        cout << "Enter payment amount: ";
        cin >> payment;

        if (payment > 0 && payment <= feeBalance) {
            feeBalance -= payment;
            cout << "Payment successful.\n";
        } else if (payment > feeBalance) {
            cout << "Payment cannot exceed the fee balance.\n";
        } else {
            cout << "Invalid payment.\n";
        }
    }

    void displayStatus() {
        cout << fixed << setprecision(2);
        cout << "\n===== FEE STATUS =====\n";
        cout << "Student Name      : " << studentName << '\n';
        cout << "Admission Number : " << admissionNumber << '\n';
        cout << "Remaining Balance: " << feeBalance << '\n';
    }
};

int main() {
    Student student;

    student.inputStudent();
    student.makePayment();
    student.displayStatus();

    return 0;
}
```

Week 8 — Task 1

Bank Account System (Constructor and Destructor)

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class BankAccount {
private:
    string accountHolderName;
    string accountNumber;
    double accountBalance;

public:
    BankAccount(string name, string number, double balance) {
        accountHolderName = name;
        accountNumber = number;
        accountBalance = balance;
        cout << "Account successfully created\n";
    }

    void displayAccount() {
        cout << fixed << setprecision(2);
        cout << "\n===== ACCOUNT DETAILS =====\n";
        cout << "Account Holder : " << accountHolderName << '\n';
        cout << "Account Number : " << accountNumber << '\n';
        cout << "Account Balance: " << accountBalance << '\n';
    }

    ~BankAccount() {
        cout << "Account closed successfully\n";
    }
};

int main() {
    string name, number;
    double balance;

    cout << "Enter account holder name: ";
    getline(cin, name);

    cout << "Enter account number: ";
    getline(cin, number);

    cout << "Enter account balance: ";
    cin >> balance;

    BankAccount account(name, number, balance);
    account.displayAccount();

    return 0;
}
```

Week 8 — Task 2

Hotel Room Booking System

```
#include <iostream>
#include <string>
using namespace std;

class RoomBooking {
private:
    string guestName;
    int roomNumber;
    int numberOfDays;

public:
    RoomBooking(string name, int room, int days) {
        guestName = name;
        roomNumber = room;
        numberOfDays = days;
        cout << "Room booking confirmed\n";
    }

    void displayBooking() {
        cout << "\n===== BOOKING DETAILS =====\n";
        cout << "Guest Name      : " << guestName << '\n';
        cout << "Room Number   : " << roomNumber << '\n';
        cout << "Days Booked    : " << numberOfDays << '\n';
    }

    ~RoomBooking() {
        cout << "Booking ended, room released\n";
    }
};

int main() {
    string name;
    int room, days;

    cout << "Enter guest name: ";
    getline(cin, name);

    cout << "Enter room number: ";
    cin >> room;

    cout << "Enter number of days booked: ";
    cin >> days;

    RoomBooking booking(name, room, days);
    booking.displayBooking();

    return 0;
}
```

Week 9 — Task 1

School Management System (Inheritance)

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int age;

public:
    void inputPerson() {
        cout << "Enter name: ";
        getline(cin, name);

        cout << "Enter age: ";
        cin >> age;
    }

    void displayPerson() {
        cout << "Name : " << name << '\n';
        cout << "Age : " << age << '\n';
    }
};

class Student : public Person {
private:
    string admissionNumber;
    string studentClass;

public:
    void inputStudent() {
        inputPerson();
        cin.ignore();

        cout << "Enter admission number: ";
        getline(cin, admissionNumber);

        cout << "Enter class: ";
        getline(cin, studentClass);
    }

    void displayStudent() {
        cout << "\n===== STUDENT DETAILS =====\n";
        displayPerson();
        cout << "Admission Number: " << admissionNumber << '\n';
        cout << "Class : " << studentClass << '\n';
    }
};

int main() {
    Student student;

    student.inputStudent();
    student.displayStudent();

    return 0;
}
```

Week 9 — Task 2

Vehicle System (Inheritance)

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    string brand;
    int yearOfManufacture;

public:
    void inputVehicle() {
        cout << "Enter brand: ";
        getline(cin, brand);

        cout << "Enter year of manufacture: ";
        cin >> yearOfManufacture;
    }

    void displayVehicle() {
        cout << "Brand          : " << brand << '\n';
        cout << "Year of Manufacture : " << yearOfManufacture << '\n';
    }
};

class Car : public Vehicle {
private:
    int numberOfDoors;
    double engineCapacity;

public:
    void inputCar() {
        inputVehicle();

        cout << "Enter number of doors: ";
        cin >> numberOfDoors;

        cout << "Enter engine capacity (cc): ";
        cin >> engineCapacity;
    }

    void displayCar() {
        cout << "\n===== CAR DETAILS =====\n";
        displayVehicle();
        cout << "Number of Doors      : " << numberOfDoors << '\n';
        cout << "Engine Capacity(cc): " << engineCapacity << '\n';
    }
};

int main() {
    Car car;

    car.inputCar();
    car.displayCar();

    return 0;
}
```

Week 10 — Task 1

Payment System (Function Overloading)

```
#include <iostream>
#include <iomanip>
using namespace std;

class Payment {
public:
    // Cash payment
    void processPayment(int amount) {
        cout << "Cash payment final amount: " << amount << '\n';
    }

    // Card payment
    void processPayment(double amount, double transactionFee) {
        double finalAmount = amount + transactionFee;
        cout << fixed << setprecision(2);
        cout << "Card payment final amount: " << finalAmount << '\n';
    }

    // Mobile payment
    void processPayment(double amount, double fee, double discount) {
        double finalAmount = amount + fee - discount;
        cout << fixed << setprecision(2);
        cout << "Mobile payment final amount: " << finalAmount << '\n';
    }
};

int main() {
    Payment payment;

    payment.processPayment(1000);
    payment.processPayment(1000.0, 50.0);
    payment.processPayment(1000.0, 30.0, 100.0);

    return 0;
}
```

Week 10 — Task 2

Shape Drawing System (Function Overriding)

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() {
        cout << "Drawing Shape\n";
    }

    virtual ~Shape() {}
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing Rectangle\n";
    }
};

class Triangle : public Shape {
public:
    void draw() override {
        cout << "Drawing Triangle\n";
    }
};

int main() {
    Shape* shape;

    Circle circle;
    Rectangle rectangle;
    Triangle triangle;

    shape = &circle;
    shape->draw();

    shape = &rectangle;
    shape->draw();

    shape = &triangle;
    shape->draw();

    return 0;
}
```

Week 11 — Task 1

Employee Data Protection System (Encapsulation)

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Employee {
private:
    string employeeName;
    string employeeID;
    double basicSalary;

public:
    void setEmployeeDetails() {
        cout << "Enter employee name: ";
        getline(cin, employeeName);

        cout << "Enter employee ID: ";
        getline(cin, employeeID);

        cout << "Enter basic salary: ";
        cin >> basicSalary;
    }

    double calculateSalary() {
        return basicSalary + (0.10 * basicSalary);
    }

    void displayEmployeeDetails() {
        cout << fixed << setprecision(2);
        cout << "\n===== EMPLOYEE DETAILS =====\n";
        cout << "Employee Name : " << employeeName << '\n';
        cout << "Employee ID   : " << employeeID << '\n';
        cout << "Basic Salary  : " << basicSalary << '\n';
        cout << "Net Salary   : " << calculateSalary() << '\n';
    }
};

int main() {
    Employee employee;

    employee.setEmployeeDetails();
    employee.displayEmployeeDetails();

    return 0;
}
```

Week 11 — Task 2

Student Fee Security System (Encapsulation)

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Student {
private:
    string studentName;
    string admissionNumber;
    double feeBalance;

public:
    void inputStudentData() {
        cout << "Enter student name: ";
        getline(cin, studentName);

        cout << "Enter admission number: ";
        getline(cin, admissionNumber);

        cout << "Enter fee balance: ";
        cin >> feeBalance;
    }

    void makePayment() {
        double payment;

        cout << "Enter payment amount: ";
        cin >> payment;

        if (payment > 0 && payment <= feeBalance) {
            feeBalance -= payment;
            cout << "Payment successful.\n";
        } else if (payment > feeBalance) {
            cout << "Payment cannot exceed the fee balance.\n";
        } else {
            cout << "Invalid payment.\n";
        }
    }

    void displayStatus() {
        cout << fixed << setprecision(2);
        cout << "\n===== STUDENT FEE STATUS =====\n";
        cout << "Student Name      : " << studentName << '\n';
        cout << "Admission Number : " << admissionNumber << '\n';
        cout << "Remaining Balance: " << feeBalance << '\n';
    }
};

int main() {
    Student student;

    student.inputStudentData();
    student.makePayment();
    student.displayStatus();

    return 0;
}
```

Week 12 — Task 1

Hospital Management System (Abstraction)

Note: The assignment does not specify actual salary values, so sample values are used to demonstrate the abstraction and runtime behavior.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class MedicalStaff {
protected:
    string staffName;
    string staffID;

public:
    MedicalStaff(string name, string id)
        : staffName(name), staffID(id) {}

    virtual double calculateSalary() = 0;

    void displayDetails() {
        cout << "Staff Name: " << staffName << '\n';
        cout << "Staff ID : " << staffID << '\n';
    }

    virtual ~MedicalStaff() {}
};

class Doctor : public MedicalStaff {
private:
    double basicSalary;
    double allowance;

public:
    Doctor(string name, string id, double basic, double allow)
        : MedicalStaff(name, id), basicSalary(basic), allowance(allow) {}

    double calculateSalary() override {
        return basicSalary + allowance;
    }
};

class Nurse : public MedicalStaff {
private:
    double basicSalary;
    double overtimePay;

public:
    Nurse(string name, string id, double basic, double overtime)
        : MedicalStaff(name, id), basicSalary(basic), overtimePay(overtime) {}

    double calculateSalary() override {
        return basicSalary + overtimePay;
    }
};

int main() {
    Doctor doctor("Dr. Amina", "D001", 100000, 20000);
    Nurse nurse("Nurse Brian", "N001", 60000, 10000);

    MedicalStaff* staff;

    cout << fixed << setprecision(2);

    staff = &doctor;
    staff->displayDetails();
    cout << "Salary: " << staff->calculateSalary() << "\n\n";

    staff = &nurse;
    staff->displayDetails();
    cout << "Salary: " << staff->calculateSalary() << '\n';

    return 0;
}
```

Week 12 — Task 2

Shape Area System (Abstraction)

```
#include <iostream>
#include <iomanip>
using namespace std;

class Shape {
public:
    virtual double calculateArea() = 0;

    virtual ~Shape() {}
};

class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : radius(r) {}

    double calculateArea() override {
        const double PI = 3.141592653589793;
        return PI * radius * radius;
    }
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    Rectangle(double l, double w) : length(l), width(w) {}

    double calculateArea() override {
        return length * width;
    }
};

int main() {
    Circle circle(7);
    Rectangle rectangle(10, 5);

    Shape* shape;

    cout << fixed << setprecision(2);

    shape = &circle;
    cout << "Circle Area    : " << shape->calculateArea() << '\n';

    shape = &rectangle;
    cout << "Rectangle Area : " << shape->calculateArea() << '\n';

    return 0;
}
```

Week 13 — Task 1

Borrowed Books Text File

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    string title;

    cout << "Enter book title: ";
    getline(cin, title);

    // ios::app preserves existing records and adds the new title at the end.
    ofstream file("borrowed_books.txt", ios::app);

    if (!file) {
        cout << "Error: Could not open the file.\n";
        return 1;
    }

    file << title << '\n';
    file.close();

    cout << "Book title successfully stored.\n";

    return 0;
}
```

Week 13 — Task 2

Student Examination Results from Binary File

Note: results.dat must contain records written using the same structure layout for this binary-reading program to interpret them correctly.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

struct StudentResult {
    char name[100];
    char registrationNumber[50];
    double totalMarks;
};

int main() {
    ifstream file("results.dat", ios::binary);

    if (!file) {
        cout << "Error: Could not open results.dat\n";
        return 1;
    }

    StudentResult student;

    cout << "===== EXAMINATION RESULTS =====\n";

    while (file.read(reinterpret_cast<char*>(&student), sizeof(student))) {
        cout << "Name: " << student.name << '\n';
        cout << "Marks: " << student.totalMarks << "\n\n";
    }

    file.close();

    return 0;
}
```

Week 13 — Task 3

Retail Shop Daily Sales from Text File

```
#include <stdio.h>

int main() {
    FILE *file;
    double amount;
    double totalSales = 0.0;

    file = fopen("sales.txt", "r");

    if (file == NULL) {
        printf("Error: Could not open sales.txt\n");
        return 1;
    }

    while (fscanf(file, "%lf", &amount) == 1) {
        totalSales += amount;
    }

    printf("Total sales for the day: %.2f\n", totalSales);

    fclose(file);

    return 0;
}
```